Superstore Sales

Data Analysis and Visualization

Project Documentation: Data Analysis Certificate Task

Name: Ashrik kayastha

Gmail: ashrik.kayastha@gmail.com

Batch: DA/DS mrng jestha.

item	Details
Dataset	Superstore raw.csv(9,994 order lines, 21 columns)
Tools	Python, pandas, Numpy, Matplotlib, Seaborn,Scikit-learn, jupyter Notebook
Notebook	Analysis.ipynb

Table of content:

Contents:

1. Introduction 5

1.1 Loading the dataset 5

1.3 Summary statistics 6

1.4 Spotting potential problems 7

2. Data Cleaning and Transformation 9

2.1 Fixing data type and inconsistent text 9

2.2 Handling missing values 10

2.3 Outlier detection 11

2.4 Outlier treatment 12

2.5 Feature engineering 13

2.6 Saving the cleaned dataset 14

2.8 Scaling numerical features 16

3. Exploratory Data Analysis (EDA) 17

3.1 Descriptive statistics 17

3.2 Distributions 17

3.4 Correlation analysis 20

3.5 Trends over time 21

3.6 Category, product, region, segment and shipping analysis 21

4. Key Insights 23

5. Recommendations 24

Conclusion 25

Table of Figure:

Figure 1: Loading dataset..... 5
Figure 2: Column groups in the raw dataset..... 6
Figure 3: Summary statistics 6
Figure 4: Summary statistics of the main numeric columns. 7
Figure 5: spot potential code for missing values, duplicates. 7
Figure 6: Data check 8
Figure 7: fixing data type and inconsistent text. 9
Figure 8: Handling missing values 10
Figure 9: Handling missing values. 10
Figure 10: Outlier detection code. 11
Figure 11: Box plot for Sales, Quantity, Discount, profit. 12
Figure 12: Outlier treatment 12
Figure 13: figure after outlier. 13
Figure 14: Features. 14
Figure 15: new features. 14
Figure 16: Cleaned datasets. 14
Figure 17: Encoding categorical variables. 15
Figure 18: columns after encoding. 15
Figure 19: Scaling numeical feature. 16
Figure 20: Scaling numerical feature 16
Figure 21: Sales, profit and margin by category. 17
Figure 22: Distribution 17
Figure 23: Distribution of Numerical Feature After Data preprocssing. 18
Figure 24: sales vs profit scatter plot. 19
Figure 25: Discount vs profit (boxplot and bar chart) 19
Figure 26: Pair plot. 20
Figure 27: correlation analysis. 20
Figure 28: correlation matrix 21
Figure 29: Yearly sales, profit and growth. 21
Figure 30: profit by sub category. 22

Figure 31: Sub cat by heatmap. 22
Figure 32: Key figures. 23

1. Introduction

This document explains the process followed in the notebook to analyse the Superstore sales dataset. The goal is to clean and transform the raw data, explore it with statistics and visualizations, and turn the results into insights that can guide business decisions.

The work follows 5 stages.

1. Data collection and Exploration: Load the data and understand its structure and problems
2. Data Cleaning and Transformation: fix problems, treat outliers, create features, encode and scale.
3. Exploratory Data Analysis (EDA): summarise the data and study patterns with charts.
4. Key insights: Calculate the main business findings.
5. Recommendations and feature Selection: suggest actions and choose useful features for modelling.

1. Data Collection and Exploration

1.1 Loading the dataset

I loaded the dataset into a Pandas DataFrame using pd.read_csv(). The file is not saved in UTF-8 so I specified an encoding of latin-1; without specifying the encoding Pandas stops with a decoding error. I used df.head() to preview the rows.

```
# 1.1 Load the dataset with pandas

df = pd.read_csv('superstore_raw.csv', encoding='latin-1') #
df.head()
```

Figure 1: Loading dataset

1.2 Structure of the data

The dataset has 9,994 rows and 21 columns. Each row is one product line within a customer order. The columns fall into five groups



	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bookcases	
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454	Furniture	Chairs	
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240	Office Supplies	Labels	
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Tables	
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760	Office Supplies	Storage	

Figure 2: Column groups in the raw dataset.

Every column has 9,994 values that're not empty. Two issues were found with the data types. The dates such, as Order Date and Ship Date are saved as text. The Postal Code is saved as a number.

1.3 Summary statistics

Summary statistics were produced with describe() for the numeric and the text columns.

```
# 1.3  Summary statistics for numeric and text columns

display(df.describe().round(2))
display(df.describe(include='object'))
```

Figure 3: Summary statistics

	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.00	9994.00	9994.00	9994.00	9994.00	9994.00
mean	4997.50	55190.38	229.86	3.79	0.16	28.66
std	2885.16	32063.69	623.25	2.23	0.21	234.26
min	1.00	1040.00	0.44	1.00	0.00	-6599.98
25%	2499.25	23223.00	17.28	2.00	0.00	1.73
50%	4997.50	56430.50	54.49	3.00	0.20	8.67
75%	7495.75	90008.00	209.94	5.00	0.20	29.36
max	9994.00	99301.00	22638.48	14.00	0.80	8399.98

Figure 4: Summary statistics of the main numeric columns.

1.4 Spotting potential problems

A series of simple checks was run to find problems in the data

```
# 1.4 Spot potential problems
# checks: missing values, duplicates,impossible value.
# Check 1:
print("Missing values in each column:")
print(df.isnull().sum()) # checks missing values

# Check 2:
print("\nDuplicate rows:", df.duplicated().sum()) # Checks duplicate rows

# Check 3: checks impossible values
print("\nNegative sales:", (df["Sales"] < 0).sum())
print("Quantity of zero or less:", (df["Quantity"] <= 0).sum())
print("Discount outside 0 to 1:", ((df["Discount"] < 0) | (df["Discount"] > 1)).sum())

# Check 4: checks how many orders lost money
print("\nOrders with negative profit:", (df["Profit"] < 0).sum())
```

Figure 5: spot potential code for missing values, duplicates.

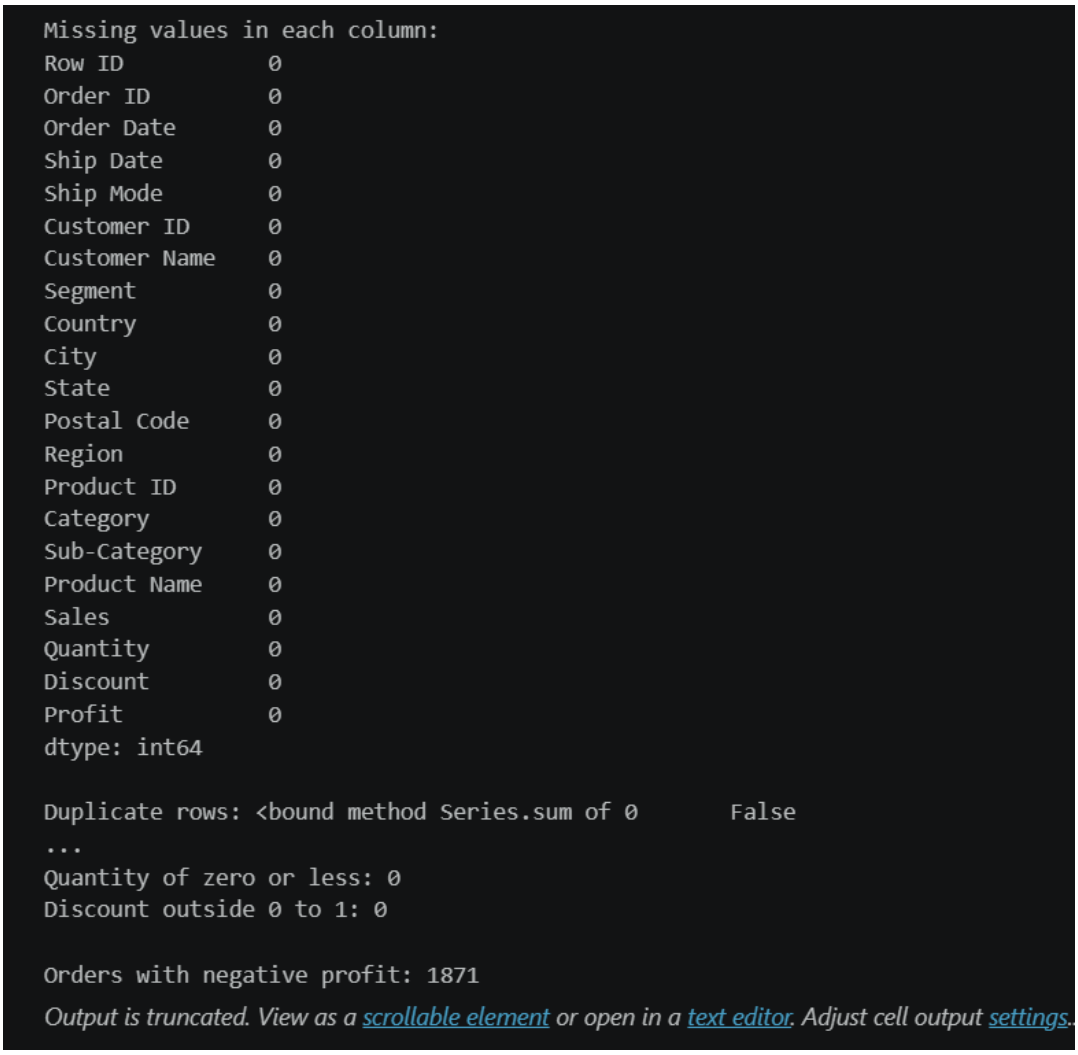


Figure 6: Data check

The data quality checks found no missing values in any of the 21 columns. No sales were negative. No quantity was zero or less. No discounts were, outside the range of 0 to 1. A total of 1,871 orders have a profit. These orders are loss-making orders, not errors so they were kept for analysis.

2. Data Cleaning and Transformation

2.1 Fixing data type and inconsistent text

```
# Strip whitespace from all text columns

# 2.1 Fix data types and inconsistent text

# Step 1: Remove extra spaces from all text columns
text_cols = df.select_dtypes(include=["object", "string"]).columns
for c in text_cols:
    df[c] = df[c].str.strip()

# Step 2: Convert dates from text to real dates (month/day/year)
df["Order Date"] = pd.to_datetime(
    df["Order Date"],
    format="%m/%d/%Y"
)
df["Ship Date"] = pd.to_datetime(
    df["Ship Date"],
    format="%m/%d/%Y"
)

# Step 3: Fix postal codes
df["Postal Code"] = df["Postal Code"].astype(str).str.zfill(5) # m

# Step 4: Remove duplicate rows
before = len(df)
df = df.drop_duplicates().reset_index(drop=True)
print("Duplicates removed:", before - len(df))

# Step 5: Check that the changes worked
print(df.dtypes)
```

Figure 7: fixing data type and inconsistent text.

This code. Standardizes the dataset by removing extra spaces and turning date columns into proper date format. The code formats codes to five digits and removes

duplicate rows from the dataset. the code checks the data types to confirm that the cleaning was successful.

2.2 Handling missing values

Missing values were checked and handled using median for numeric data and mode for text data.

Result: No rows or values were missing, so no changes were made to the dataset.

```
key_cols = ["Order ID", "Order Date", "Customer ID", "Product ID"]
n_before = len(df)
df = df.dropna(subset=key_cols)

num_cols_all = df.select_dtypes(include="number").columns.drop("Row ID") # finds numerical columns
cat_cols_all = df.select_dtypes(include=["object", "string"]).columns # finds categorical data/text
n_imputed = int(
    df[num_cols_all].isna().sum().sum() # cunts missing values in numerical and text columns
    + df[cat_cols_all].isna().sum().sum()
)
df[num_cols_all] = df[num_cols_all].fillna( # fill missing values with the median
    df[num_cols_all].median()
)
for c in cat_cols_all: #for missing categorical values
    if df[c].isna().any():
        df[c] = df[c].fillna(df[c].mode()[0])

print(
    f"Rows dropped for missing keys: {n_before - len(df)} | "
    f"cells imputed: {n_imputed}"
)

print(
    "Remaining missing values:",
    int(df.isna().sum().sum())
)
```

Figure 8: Handling missing values

```
Rows dropped for missing keys: 0 | cells imputed: 0
Remaining missing values: 0
```

Figure 9: Handling missing values.

2.3 Outlier detection

Outliers were detected using the IQR and z-score methods, and box plots were used to visually check and confirm the unusual values.

2.3 Outlier detection: box plots + IQR rule

```
outlier_cols = ["Sales", "Quantity", "Discount", "Profit"]

def iqr_bounds(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - k * iqr, q3 + k * iqr

rows = []
for c in outlier_cols:
    lo, hi = iqr_bounds(df[c])
    z = (df[c] - df[c].mean()) / df[c].std()
    rows.append({
        "column": c, "lower_bound": round(lo, 2), "upper_bound": round(hi, 2),
        "IQR outliers": int(((df[c] < lo) | (df[c] > hi)).sum()),
        "% of rows": round(((df[c] < lo) | (df[c] > hi)).mean() * 100, 1),
        "|z|>3 outliers": int((z.abs() > 3).sum()),
    })
outlier_summary = pd.DataFrame(rows).set_index("column")
outlier_summary
```

24]

column	lower_bound	upper_bound	IQR outliers	% of rows	|z|>3 outliers
Sales	-271.71	498.93	1167	11.7	127
Quantity	-2.50	9.50	170	1.7	113
Discount	-0.30	0.50	856	8.6	300
Profit	-39.72	70.82	1881	18.8	107

Figure 10: Outlier detection code.

```python
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, c in zip(axes, outlier_cols):
    sns.boxplot(y=df[c], ax=ax, color="#9ecae1", fliersize=2)
    ax.set_title(f"{c} (before treatment)")

plt.show()
```

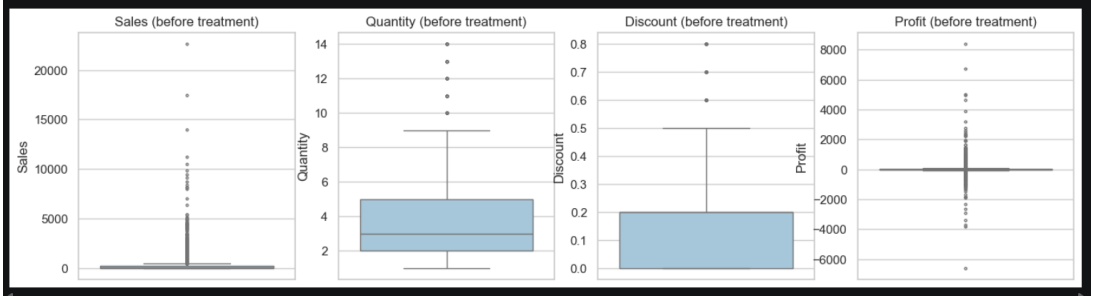


Figure 11: Box plot for Sales, Quantity, Discount, profit.

2.4 Outlier treatment

An outlier is a data value that is very different from most other values in a dataset.

```python
for c in ["Sales", "Profit"]:
    lo, hi = iqr_bounds(df[c])
    df[f"{c}_Outlier"] = ((df[c] < lo) | (df[c] > hi)).astype(int) # this create a new
    p1, p99 = df[c].quantile([0.01, 0.99])
    df[f"{c}_Winsor"] = df[c].clip(p1, p99) # winsorization reduce extrem values

df["Log_Sales"] = np.log1p(df["Sales"])
print("Skew before -> after treatment")
print(pd.DataFrame({
    "Sales": [df["Sales"].skew(), df["Sales_Winsor"].skew(), df["Log_Sales"].skew()],
    "Profit": [df["Profit"].skew(), df["Profit_Winsor"].skew(), np.nan],
}, index=["original", "winsorized (1-99%)", "log1p"]).round(2))
```

```
Skew before -> after treatment
                    Sales  Profit
original            12.97    7.56
winsorized (1-99%)   3.53    2.30
log1p                0.28     NaN
```

Figure 12: Outlier treatment

This code finds outliers in the Sales and Profit columns. It marks those outliers. Then uses Winsorization to reduce extreme values. It also applies a log transformation to the Sales column. After these steps it compares the skewness of the data before and after the treatment. This helps to see how better the data distribution has become.



Figure 13: figure after outlier.

This figure is created to visually check the data after outlier treatment**.** It shows three box plots for Winsorized Sales, Winsorized Profit, and Log-transformed Sales, helping us see whether extreme values and skewness have been reduced.

2.5 Feature engineering

New columns were created from domain in knowledge to enrich the dataset.

New feature	How it is built	Purpose
Order Year, Month, Quarter, Day of Week, Year-Month	Taken from Order Date	Trends and seasonality
Shipping Days	Ship Date − Order Date	To measure delivery speed
Profit Margin	Profit ÷ Sales	To measure profitability
Unit Price	Sales ÷ Quantity after discount	Sales ÷ Quantity after discount
Discount Band	Groups discounts into levels	To compare discount levels
Is Loss	1 if profit is negative, otherwise 0	To identify loss-making sales

Order Size	Number of items in an order	To measure basket size
Customer Orders	Number of orders made by a customer	To understand customer loyalty

```python
df["Order Year"] = df["Order Date"].dt.year
df["Order Month"] = df["Order Date"].dt.month
df["Order Quarter"] = df["Order Date"].dt.quarter
df["Order Day of Week"] = df["Order Date"].dt.day_name()
df["Year-Month"] = df["Order Date"].dt.to_period("M").astype(str)
df["Shipping Days"] = (df["Ship Date"] - df["Order Date"]).dt.days
df["Profit Margin"] = df["Profit"] / df["Sales"]
df["Unit Price"] = df["Sales"] / (df["Quantity"] * (1 - df["Discount"]))
df["Discount Band"] = pd.cut(
    df["Discount"], bins=[-0.001, 0, 0.2, 0.4, 1.0],
    labels=["No discount", "Low (1-20%)", "Medium (21-40%)", "High (>40%)"],
)
df["Is Loss"] = (df["Profit"] < 0).astype(int)
df["Order Size"] = df.groupby("Order ID")["Row ID"].transform("count")
df["Customer Orders"] = df.groupby("Customer ID")["Order ID"].transform("nunique")
df[["Order Year", "Shipping Days", "Profit Margin", "Unit Price", "Discount Band", "Is Loss", "Order Size", "Customer Orders"]].head()
```

Figure 14: Features.

	Order Year	Shipping Days	Profit Margin	Unit Price	Discount Band	Is Loss	Order Size	Customer Orders
0	2016	3	0.1600	130.98	No discount	0	2	3
1	2016	3	0.3000	243.98	No discount	0	2	3
2	2016	4	0.4700	7.31	No discount	0	1	5
3	2015	7	-0.4000	348.21	High (>40%)	1	2	6
4	2015	7	0.1125	13.98	Low (1-20%)	0	2	6

Figure 15: new features.

2.6 Saving the cleaned dataset

2.6 Save the cleaned dataset (reafable, un-encoded)

```python
df.to_csv(PROCESSED_DIR / "superstore_cleaned.csv", index=False)
print("Saved:", PROCESSED_DIR / "superstore_cleaned.csv", df.shape)
```

```
Saved: data\processed\superstore_cleaned.csv (9994, 38)
```

Figure 16: Cleaned datasets.

This code saves the cleaned dataset as superstore_cleaned.csv in the processed folder. It also prints the file location and the number of rows and columns in the dataset.

2.7 Encoding categorical variables

Machine-learning methods need numbers, so text categories were encoded on a separate copy of the data, leaving the original unchanged.

```
2.7 Encoding categorical variables

    model_df = df.copy() # creates a seprate copy so orginal df remains unchanged

    onehot_cols = ["Ship Mode", "Segment", "Region", "Category"]
    model_df = pd.get_dummies(model_df, columns=onehot_cols, drop_first=False, dtype=int) # this

    for c in ["Sub-Category", "State"]:
        model_df[f"{c}_Encoded"] = LabelEncoder().fit_transform(model_df[c]) # This converts tex

    band_order = {"No discount": 0, "Low (1-20%)": 1, "Medium (21-40%)": 2, "High (>40%)": 3} #
    model_df["Discount Band"] = model_df["Discount Band"].astype(str).map(band_order)

    print("Columns after encoding:", model_df.shape[1])
    model_df.filter(regex="^(Ship Mode_|Segment_|Region_|Category_)|_Encoded$").head()
9]
```

Figure 17: Encoding categorical variables.

```
Columns after encoding: 50
```

	Ship Mode_First Class	Ship Mode_Same Day	Ship Mode_Second Class	Ship Mode_Standard Class	Segment_Consumer	Segment_Corporate	Segment_Home Office	Region_Central	Region_East	Region_South	Region
0	0	0	1	0	1	0	0	0	0	1	
1	0	0	1	0	1	0	0	0	0	1	
2	0	0	1	0	0	1	0	0	0	0	
3	0	0	0	1	1	0	0	0	0	1	
4	0	0	0	1	1	0	0	0	0	1	

Figure 18: columns after encoding.

Encoding converts categorical text data into numerical values for machine learning. Here, categories were converted into 0/1 values or numerical labels, and the Discount Band was converted into ordered values from 0 to 3**.** A separate copy was used to keep the original dataset unchanged.

2.8 Scaling numerical features

Scaling is the process of making numerical features have a range so that no one feature has too much impact on the model.

Here we used StandardScaler to scale features like Sales, Profit, Quantity, Discount and Shipping Days. We then removed ID and text columns and saved the final model-ready dataset, as a CSV file.

```
2.8 Scaling numerical features

    to_scale = ["Sales_Winsor", "Log_Sales", "Quantity", "Discount", "Profit_Winsor",
                "Profit Margin", "Unit Price", "Shipping Days", "Order Size", "Customer Orders"]
    scaler = StandardScaler()
    scaled = pd.DataFrame(scaler.fit_transform(model_df[to_scale]),
                          columns=[f"{c}_Scaled" for c in to_scale], index=model_df.index)
    model_df = pd.concat([model_df, scaled], axis=1)
    print(scaled.describe().loc[["mean", "std"]].round(3))

    # Save the model-ready dataset (identifiers/text dropped, everything numeric)
    drop_for_model = ["Row ID", "Order ID", "Order Date", "Ship Date", "Customer ID", "Customer
                      "Country", "City", "State", "Postal Code", "Product ID", "Product Name",
                      "Sub-Category", "Order Day of Week", "Year-Month"]
    model_ready = model_df.drop(columns=drop_for_model)
    model_ready.to_csv(PROCESSED_DIR / "superstore_model_ready.csv", index=False)
    print("Saved:", PROCESSED_DIR / "superstore_model_ready.csv", model_ready.shape)
```

Figure 19: Scaling numeical feature.

```
      Sales_Winsor_Scaled  Log_Sales_Scaled  Quantity_Scaled  Discount_Scaled  \
mean                  0.0               0.0              0.0             -0.0
std                   1.0               1.0              1.0              1.0

      Profit_Winsor_Scaled  Profit Margin_Scaled  Unit Price_Scaled  \
mean                  -0.0                   0.0               -0.0
std                    1.0                   1.0                1.0

      Shipping Days_Scaled  Order Size_Scaled  Customer Orders_Scaled
mean                  -0.0               -0.0                    -0.0
std                    1.0                1.0                     1.0
Saved: data\processed\superstore_model_ready.csv (9994, 45)
```

Figure 20: Scaling numerical feature .

3. Exploratory Data Analysis (EDA)

3.1 Descriptive statistics

Descriptive statistics were computed for the columns, such as Profit Margin and Shipping Days. For every column the count of rows the proportion of rows the total Sales, the total Profit and the profit margin were also recorded.

```
num_summary = df[["Sales", "Quantity", "Discount", "Profit", "Profit Margin", "Shipping Days"]].describe().T
num_summary["skew"] = df[["Sales", "Quantity", "Discount", "Profit", "Profit Margin", "Shipping Days"]].skew()
print(num_summary.round(2))
```

```
               count    mean     std      min    25%    50%     75%  \
Sales         9994.0  229.86  623.25     0.44  17.28  54.49  209.94
Quantity      9994.0    3.79    2.23     1.00   2.00   3.00    5.00
Discount      9994.0    0.16    0.21     0.00   0.00   0.20    0.20
Profit        9994.0   28.66  234.26 -6599.98   1.73   8.67   29.36
Profit Margin 9994.0    0.12    0.47    -2.75   0.08   0.27    0.36
Shipping Days 9994.0    3.96    1.75     0.00   3.00   4.00    5.00

                   max   skew
Sales          22638.48  12.97
Quantity          14.00   1.28
Discount           0.80   1.68
Profit          8399.98   7.56
Profit Margin      0.50  -2.89
Shipping Days      7.00  -0.42
```

Figure 21: Sales, profit and margin by category.

3.2 Distributions

Histograms with density curves were created for Sales log Sales, winsorized Profit, Quantity, Discount and Profit Margin. A density plot showed a comparison of log Sales, across the three categories.

- Sales and Profit are concentrated near small values with a long right tail; after the log transform, Sales looks close to a bell shape.
- Most orders have a Quantity of 2 to 5 units, and Discount is concentrated at 0 and 0.20.
- Profit Margin has most values between 0 and 0.4, with a long tail of strongly negative margins (loss-making lines).

3.2 Distribution (histograms and density plots)

```
fig, axes = plt.subplots(2, 3, figsize=(16, 8))
sns.histplot(df["Sales_Winsor"], bins=50, kde=True, ax=axes[0, 0], color="#3182bd").set_title("Sales (winsorized)")
sns.histplot(df["Log_Sales"], bins=50, kde=True, ax=axes[0, 1], color="#31a354").set_title("log(1 + Sales)")
sns.histplot(df["Profit_Winsor"], bins=60, kde=True, ax=axes[0, 2], color="#e6550d").set_title("Profit (winsorized)")
sns.countplot(x=df["Quantity"], ax=axes[1, 0], color="#6baed6").set_title("Quantity")
sns.countplot(x=df["Discount"], ax=axes[1, 1], color="#9e9ac8").set_title("Discount")
axes[1, 1].tick_params(axis="x", rotation=60)
sns.histplot(df["Profit Margin"].clip(-1.5, 0.6), bins=50, kde=True, ax=axes[1, 2], color="#fd8d3c").set_title("Profit Margin (clipped for display)")
save_fig("03_distributions")
```

Figure 22: Distribution .

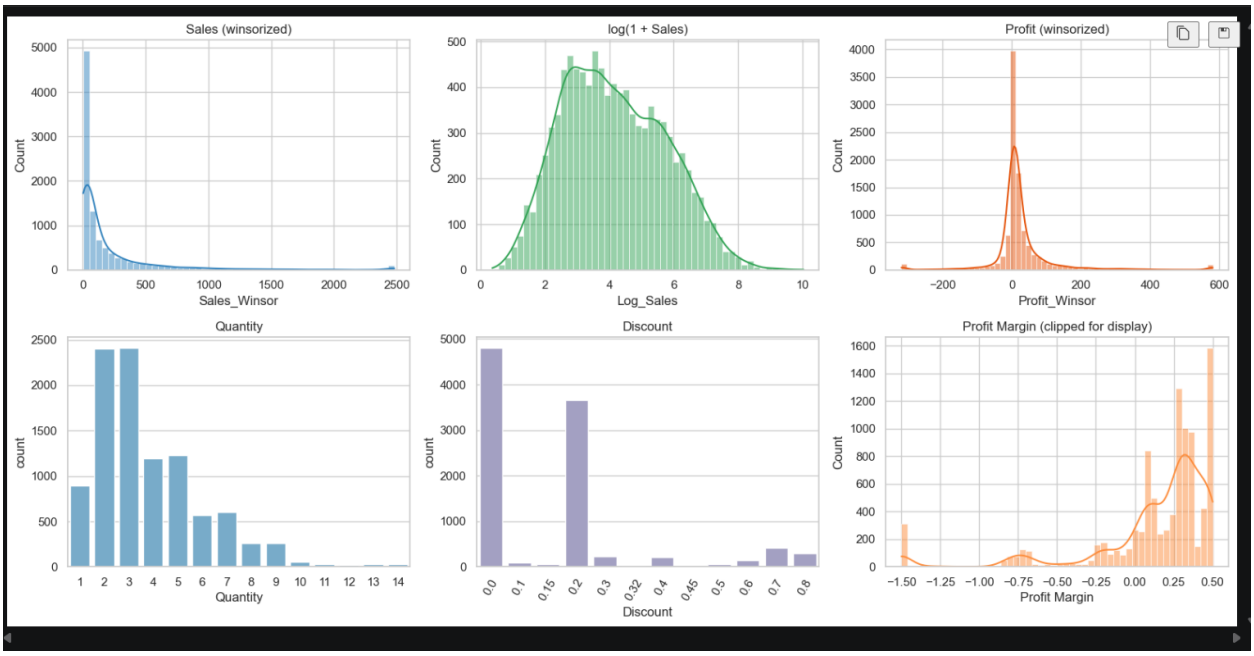


Figure 23: Distribution of Numerical Feature After Data preprocessing.

This figure shows the distribution of important numerical features such as Sales, Profit, Quantity, Discount, and Profit Margin**.** It helps us understand how the values are distributed in the dataset. The graphs also help identify unusual values and patterns in the data. Overall, it shows the effect of outlier treatment and data transformation**.**

3.3 Relationships between variables

- **Sales vs Profit scatter plot:** larger sales tend to give larger profit, but with a wide spread and many loss-making points, so high sales do not guarantee profit.
- **Discount vs Profit (box plot and bar chart):** profit falls as discount rises. Average profit is positive at discounts up to 20%, and negative from 30% upward.
- **Pair plot (1,500-row sample):** confirms the Sales-Profit link and the negative effect of high discounts.

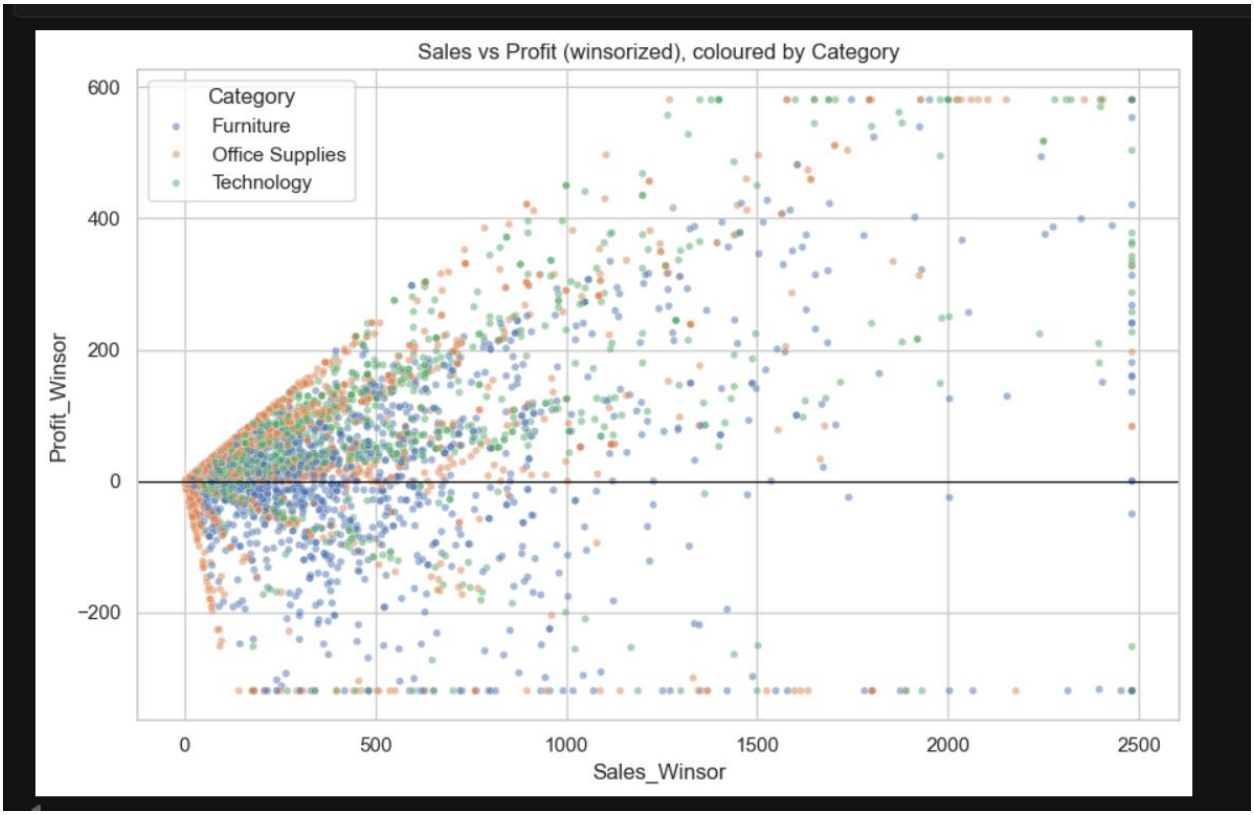


Figure 24: sales vs profit scatter plot.

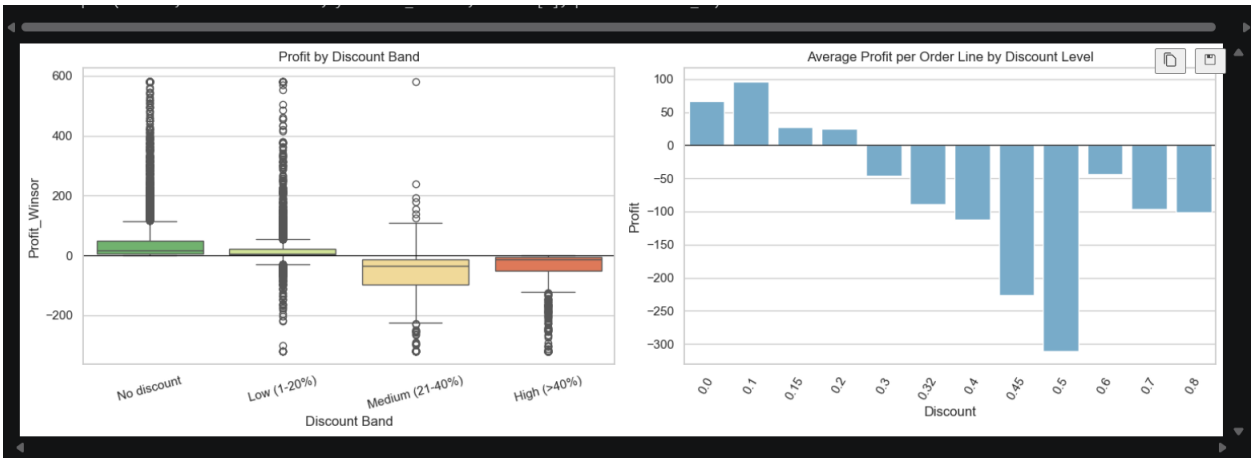


Figure 25: Discount vs profit (boxplot and bar chart)

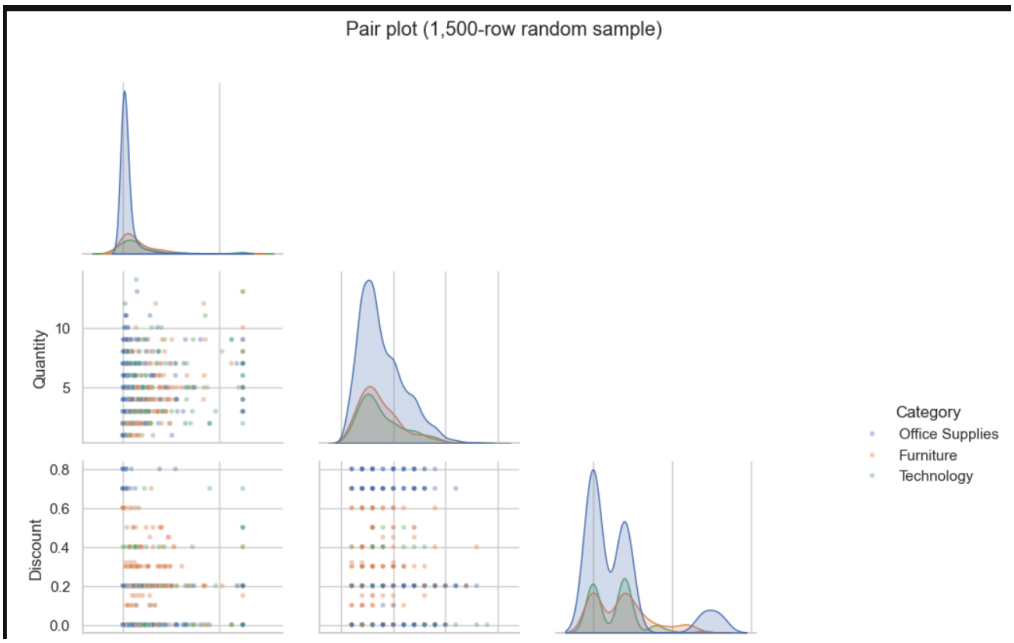


Figure 26: Pair plot.

3.4 Correlation analysis

This heatmap shows the relationships between numerical features. It helps identify positive and negative correlations**,** such as the relationship between Sales, Profit, Discount, and Profit Margin**.**

```
corr_cols = ["Sales", "Quantity", "Discount", "Profit", "Profit Margin", "Unit Price",
             "Shipping Days", "Order Size", "Customer Orders", "Is Loss"]
corr = df[corr_cols].corr(method="pearson")
plt.figure(figsize=(10, 8))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="coolwarm", center=0, square=True, linewidths=0.5)
plt.title("Correlation matrix (Pearson)")
save_fig("08_correlation_heatmap")

# Spearman is more reliable for skewed data, so compare the key pairs
spear = df[["Sales", "Quantity", "Discount", "Profit"]].corr(method="spearman")
print("Spearman correlation (rank-based, robust to outliers):")
print(spear.round(2))
```

Figure 27: correlation analysis.

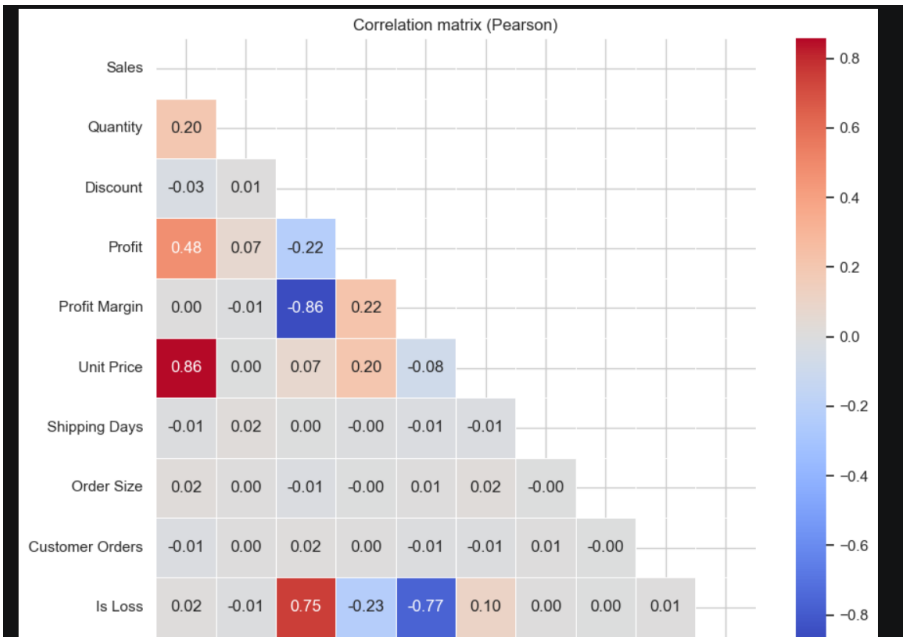


Figure 28: correlation matrix .

3.5 Trends over time

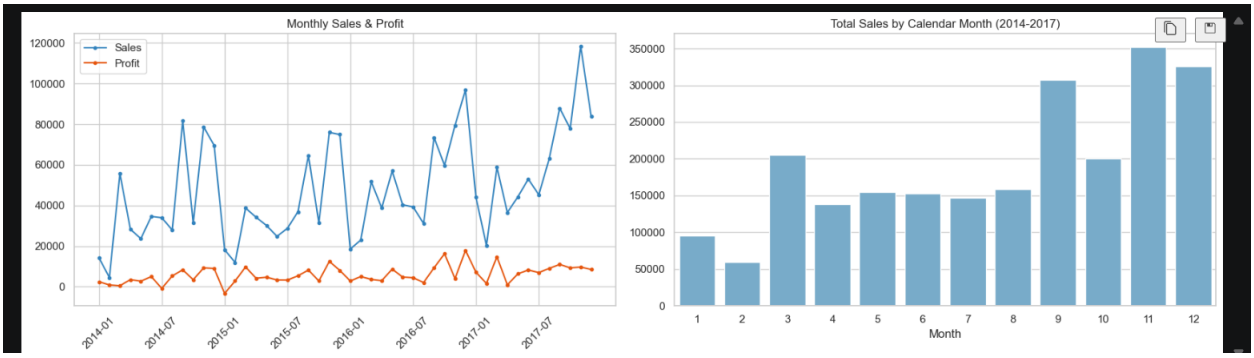


Figure 29: Yearly sales, profit and growth.

Sales grew from $484K in 2014 to $733K in 2017 and peak every year in September, November and December. Profit stays much lower and more stable than sales.

3.6 Category, product, region, segment and shipping analysis

Profit by sub-category: Copiers (about $55,600), Phones (about $44,500) and Accessories (about $41,900) earn the most profit. Tables (-$17,725), Bookcases (-$3,473) and Supplies (-$1,189) lose money overall

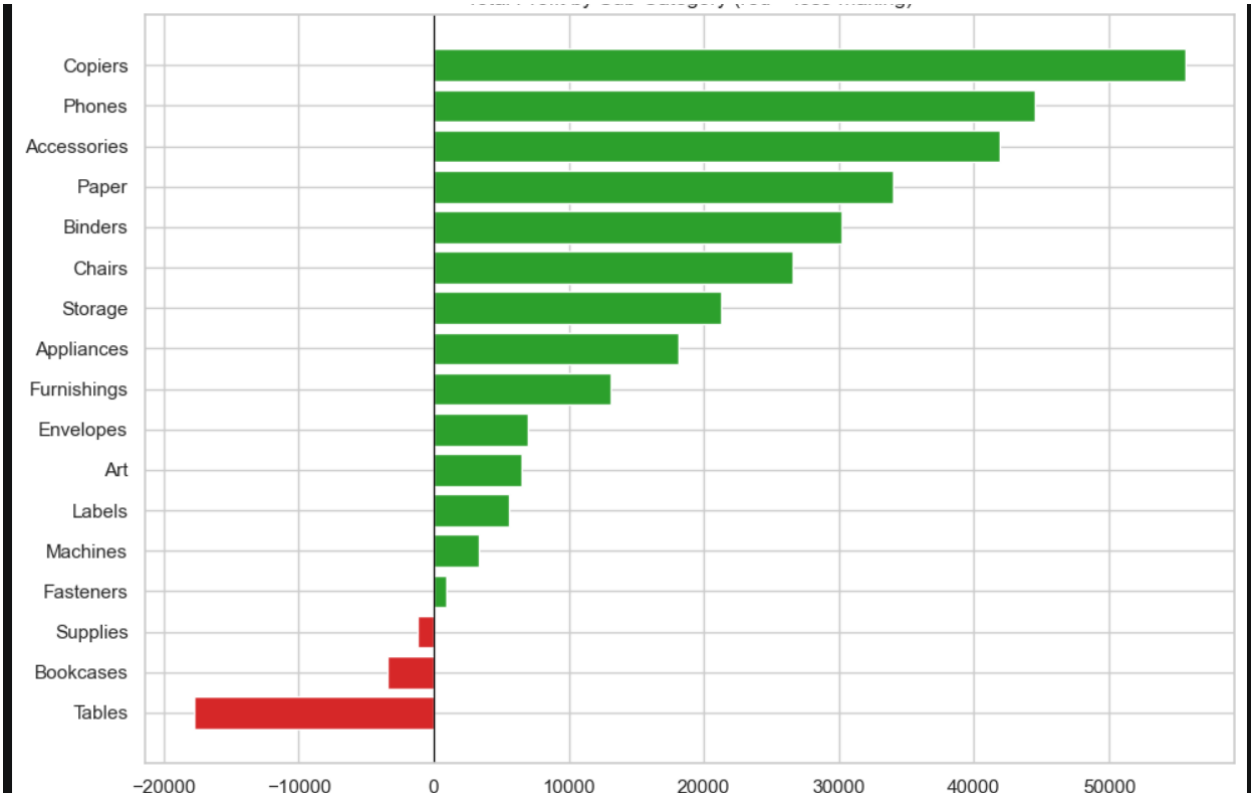


Figure 30: profit by sub category.

Sub-category by region heatmap: shows which product and region combinations make or lose money, and helps locate the problem areas.

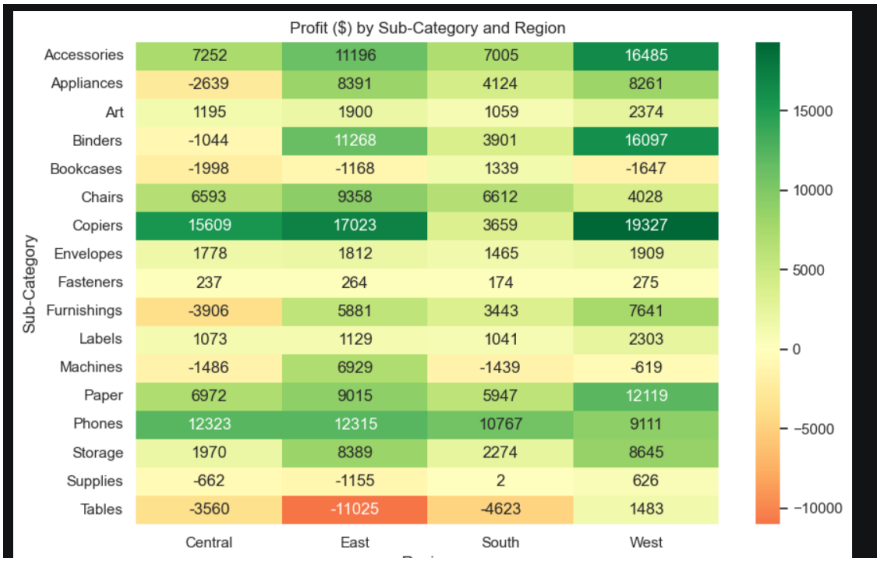


Figure 31: Sub cat by heatmap.

4. Key Insights

```
Total Sales: $2,297,201
Total Profit: $286,397
Profit Margin: 12.5%
Top 3 Sales Months: [11, 12, 9]
Top 10% Customer Sales Share: 30.6%
Loss-making Lines: 18.7%

Most Profitable States:
State
Washington    33402.6517
New York      74038.5486
California    76381.3871
Name: Profit, dtype: float64

Least Profitable States:
State
Texas          -25729.3563
Ohio           -16971.3766
Pennsylvania   -15559.9603
Name: Profit, dtype: float64
```

Figure 32: Key figures.

Main findings

- **Sales Growth:** Sales increased strongly in 2016 and 2017, after a small decrease in 2015.
- **Discounts and Losses:** Higher discounts lead to lower profit. Discounts above 20% cause many sales to make a loss.
- **Furniture:** Furniture has the lowest profit margin compared to other categories.
- **Regional Performance:** The West region has the highest profit margin, while Central has the lowest.

- **Customers:** Consumers are the largest customer group, but they have the lowest profit margin. The top 10% of customers generate about one-third of total sales.
- **Seasonal Sales:** Sales are highest around September, November, and **December**.
- **Shipping:** Standard shipping takes the longest, while Same Day is the fastest. Shipping time has little effect on profit.

5. Recommendations

- **Control discounts:** review or cap discounts above about 20%, since they almost always produce a loss
- **Fix loss-making products:** re-price or review supplier costs for Tables, Bookcases and Supplies, and improve Furniture margins overall.
- **Investigate weak regions and states:** check pricing, shipping cost and discount policy in the Central region and in Texas, Ohio and Pennsylvania.
- **Plan for peak months:** prepare stock, staff and marketing for September, November and December.
- **Protect key customers:** keep and reward the top customers who generate a large share of sales.

Final check

The final files were checked to make sure everything was correct. Both files had no missing values, and the model-ready file contained only numeric data. All checks passed successfully.

Conclusion

The data was mostly complete, but some cleaning and outlier handling were needed. After analysis, the business has an overall 12.5% profit margin, but high discounts and weak Furniture sales reduce profitability. The final cleaned and model-ready files are prepared for reporting and future prediction models**.**